ARQUITETURA DE DADOS E BIG DATA

Fundamentos da arquitetura de dados distribuídos

Módulo 2 · Overview das ferramentas para o gerenciamento de dados distribuídos

- Origem da computação distribuída (Apache)
- Sistemas gerenciadores de bases NoSQL
- Arquiteturas modernas de dados:
  - Data Lake
  - Arquiteturas de processamento
  - Lakehouse

1

BLOCO

Apache e evolução

Apache, Hadoop e o ecossistema distribuído

- A Apache Software Foundation: o guarda-chuva open source por trás de tudo
- HDFS e MapReduce: os dois pilares do Hadoop
- Princípios, ecossistema e evolução

Apache Software Foundation: O guarda-chuva open source

- Apache não é uma ferramenta única. É uma fundação sem fins lucrativos que hospeda centenas de projetos de software livre e aberto voltados à computação distribuída.
- Hadoop, Spark, Hive, Kafka, Flink, HBase e Pig, entre outros, são projetos Apache: código aberto, mantido por comunidades globais de desenvolvedores, disponível gratuitamente para uso, modificação e distribuição.
- Isso explica por que essas ferramentas se integram bem entre si: nasceram sob a mesma governança técnica e frequentemente compartilham convenções, formatos de dados e interfaces.
- Vendors comerciais (como a própria Databricks) frequentemente empacotam, testam e vendem versões gerenciadas desses projetos Apache, cobrando pelo suporte e pela facilidade de uso, não pelo software em si.

Hadoop

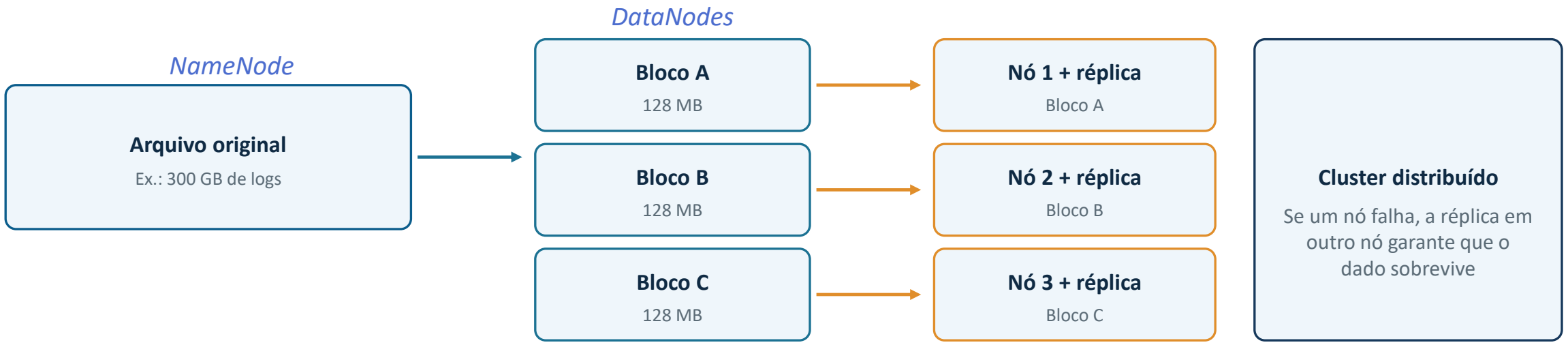
- Criado em 2005 pela Apache Foundation, o Hadoop respondeu a uma pergunta simples de enunciar e difícil de resolver: o que fazer quando os dados não cabem mais em uma única máquina, por maior e mais cara que ela seja.
- A resposta: em vez de escalar verticalmente (comprar um computador cada vez maior), escalar horizontalmente, usando um cluster de computadores comuns e baratos (commodity hardware) trabalhando em conjunto.
- Essa mudança de paradigma, de "*uma máquina poderosa*" para "*muitas máquinas comuns coordenadas*", é o princípio que sustenta praticamente toda a computação distribuída de Big Data.

ANALOGIA

Imagine procurar a carta de ouros num baralho de 52 cartas. Uma pessoa sozinha demora 1 minuto para encontrar a carta. Vinte e seis pessoas, cada uma com duas cartas, encontram a carta quase instantaneamente. Esse é o princípio por trás do Hadoop.

- Escalabilidade horizontal: para processar mais dados, basta adicionar mais máquinas comuns ao cluster, em vez de trocar por uma máquina mais potente e mais cara.
- Computação distribuída: dados e tarefas de processamento são divididos entre as máquinas do cluster, permitindo processamento verdadeiramente paralelo.
- Tolerância a falhas: a replicação de dados garante que a falha de uma máquina não derruba o processamento nem causa perda de dados; o Hadoop redireciona automaticamente as tarefas para outro nó disponível.
- Localidade de dados: sempre que possível, o processamento é levado até onde o dado já está armazenado, em vez de transportar o dado pela rede até o processamento, reduzindo drasticamente a sobrecarga de rede.

HDFS: armazenamento distribuído e replicado



- HDFS (Hadoop Distributed File System) quebra um arquivo grande em blocos (geralmente 128 MB), distribui esses blocos entre várias máquinas do cluster e mantém cópias redundantes (replicação) de cada bloco.
- Isso garante tolerância a falhas: se uma máquina para de funcionar, o dado não se perde, porque existe réplica em outro nó.
- Também garante armazenamento de baixo custo: em vez de discos especializados e caros, o HDFS roda sobre hardware comum, o que barateia drasticamente o custo por terabyte armazenado.

Hadoop: vantagens e desvantagens

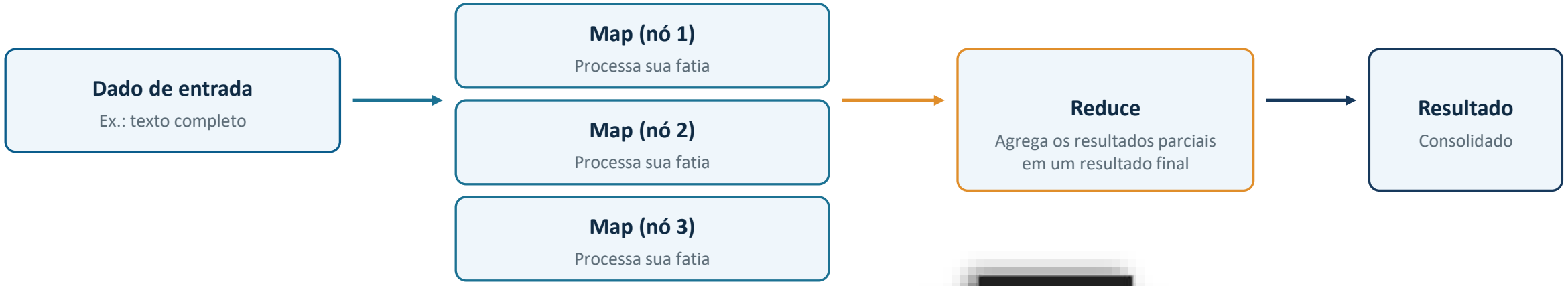
Vantagens

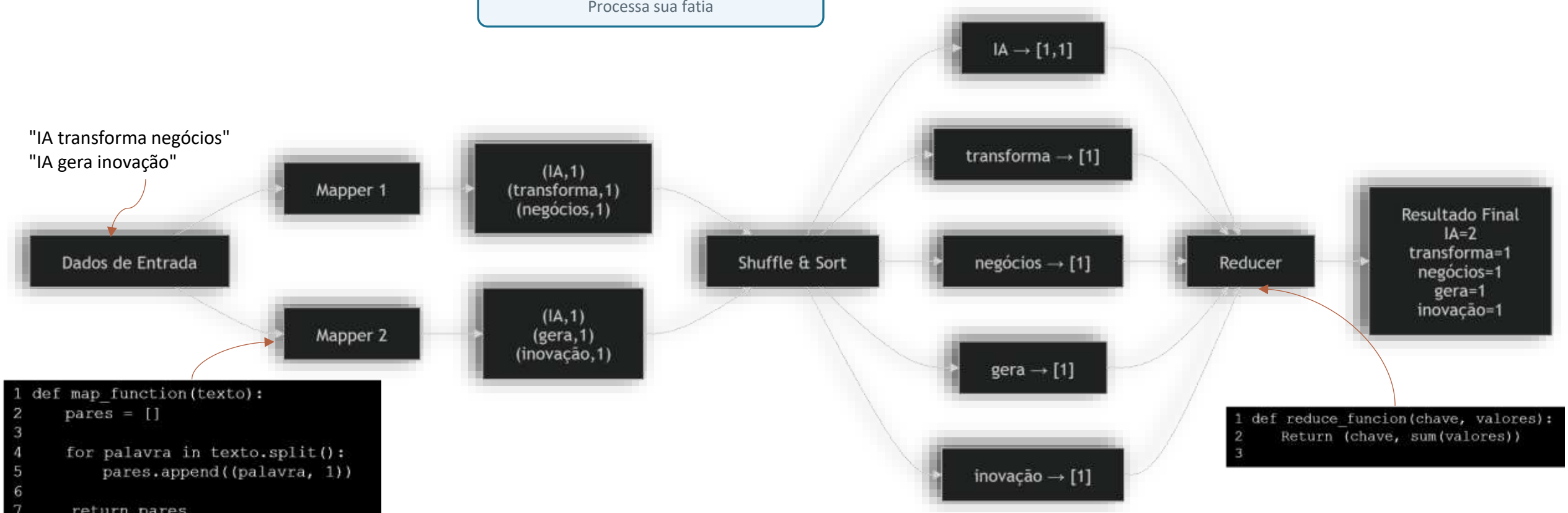
- Escalabilidade: cresce adicionando máquinas comuns ao cluster
- Tolerância a falhas via replicação automática de dados
- Custo de armazenamento baixo em hardware comum
- Flexibilidade: processa dado estruturado, semiestruturado e não estruturado
- Ecossistema amplo de ferramentas complementares
- Grande comunidade open source de suporte

Desvantagens

- Curva de aprendizado íngreme, exige conhecimento especializado em sistemas distribuídos
- Alta latência: MapReduce é batch por natureza, ruim para cenários que exigem resposta quase em tempo real
- Custo de manter um cluster físico próprio, com hardware e rede dedicados
- Sobrecarga de armazenamento pela replicação de dados
- Ecossistema complexo: integrar e manter várias ferramentas exige esforço contínuo
- Desafios adicionais de segurança e governança em ambiente distribuído

MapReduce: o modelo de programação Map e Reduce






```
1 def map_function(texto):
2     pares = []
3
4     for palavra in texto.split():
5         pares.append((palavra, 1))
6
7     return pares
```

```
1 def reduce_funcion(chave, valores):
2     Return (chave, sum(valores))
3
```

O ecossistema de ferramentas em torno do Hadoop

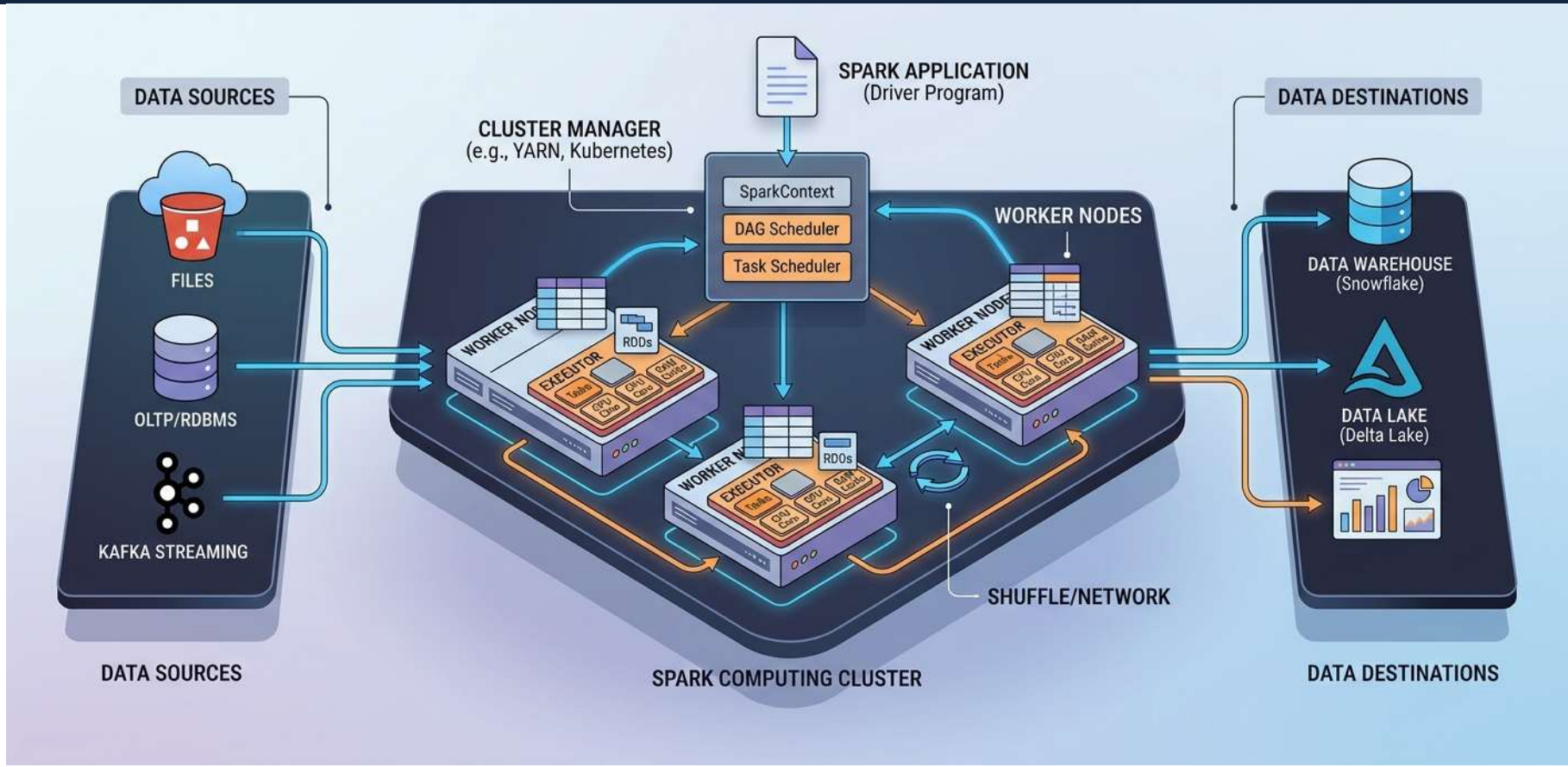
Ferramenta	Função	Analogia
YARN	Gerenciamento de recursos e escalonamento de tarefas no cluster	O "gerente de obras" que decide quem trabalha onde
HDFS	Armazenamento distribuído	Um enorme depósito com vários galpões, onde cada objeto tem uma réplica em cada galpão.
MapReduce	Processamento distribuído	Uma linha de montagem onde cada funcionario coloca uma peça.
Pig	Linguagem de script de alto nível para análise de dados	Um atalho para escrever pipelines sem programar MapReduce puro
HBase	Banco de dados NoSQL colunar que roda sobre o HDFS	Leitura e escrita rápidas de registros individuais em escala
Spark	Motor de processamento em memória, mais rápido que MapReduce	O sucessor natural do MapReduce
Hive	Data warehousing: permite consultar dados via linguagem parecida com SQL	A "tradução" de SQL para o mundo distribuído
Presto	Motor de consultas SQL distribuídas sobre múltiplas fontes	Consulta federada, sem precisar mover os dados

Problema → Tecnologia → Limitação → Nova Tecnologia→ Nova Limitação → Nova Evolução

Apache Spark

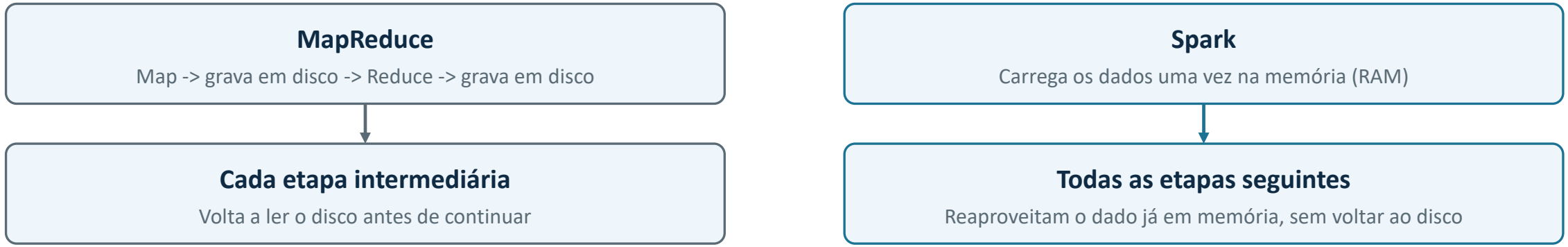
CURIOSIDADE

Alguns dos criadores originais do Spark fundaram a Databricks em 2013, transformando o projeto acadêmico em uma das plataformas comerciais mais usadas hoje em Big Data.



- Spark nasceu em 2009 no AMPLab da UC Berkeley, criado especificamente para resolver a maior dor do MapReduce: a lentidão causada pela escrita constante de resultados intermediários em disco.
- Foi disponibilizado como open-source em 2010 e se tornou projeto da Apache Software Foundation em 2013.
- A proposta central**: processar a maior parte dos dados em memória (RAM)** em vez de disco, o que o torna dramaticamente mais rápido, sobretudo em cargas iterativas, como **treinar um modelo de machine learning**, que repete o mesmo cálculo várias vezes sobre os mesmos dados.
- Diferente do MapReduce, que só faz processamento em lote, o Spark unifica processamento batch e streaming numa única ferramenta e numa única API, o que simplifica enormemente a arquitetura de dados de uma empresa.

Spark versus MapReduce: onde está o ganho de velocidade

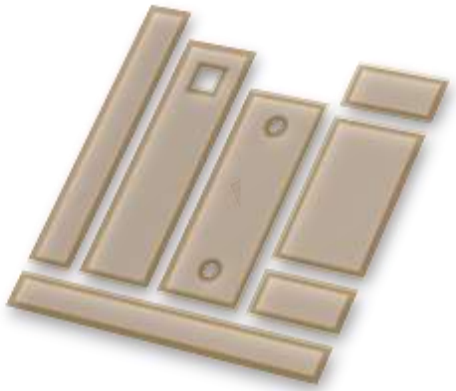


- No MapReduce, cada etapa do processamento lê do disco, processa, e grava o resultado de volta em disco antes da próxima etapa começar. Isso é seguro, mas lento, sobretudo quando o mesmo dado precisa ser processado várias vezes.
- O Spark organiza o processamento como um grafo de execução (DAG, Directed Acyclic Graph) e mantém os dados em memória entre as etapas sempre que possível, eliminando boa parte das idas e vindas ao disco.
- Resultado prático: cargas de trabalho iterativas, como algoritmos de machine learning, podem rodar dezenas de vezes mais rápido no Spark do que no MapReduce tradicional.
- O Spark também expõe APIs de alto nível em Python, Java e Scala, o que reduziu drasticamente a barreira de entrada em relação a escrever MapReduce puro.

As bibliotecas que estendem o Spark

- **MLlib**: biblioteca de machine learning distribuído, que permite treinar modelos preditivos diretamente sobre dados em escala, sem precisar extrair os dados para uma única máquina.
- **GraphX**: biblioteca para computação sobre grafos, útil para análise de redes, relacionamentos e recomendações.
- **Spark Streaming**: processamento de dados em tempo real, tratando fluxos contínuos como uma sequência de pequenos lotes (micro-batches), unificando a mesma lógica de programação usada no processamento batch.

- **Autoscaling**: ajusta dinamicamente o número de máquinas alocadas a uma tarefa conforme a demanda computacional varia, otimizando custo e desempenho ao mesmo tempo.
- **Project Tungsten**: conjunto de otimizações internas da Databricks para aproveitar melhor a arquitetura de processadores modernos, aumentando ainda mais a eficiência do motor Spark.



Atualidade

- Hadoop teve adoção maciça no início dos anos 2010, mas seu uso vem caindo à medida que a computação em nuvem amadureceu.
- A infraestrutura de nuvem hoje substitui o cluster físico do Hadoop, e o armazenamento de objetos em nuvem (Amazon S3, Azure Data Lake Storage, Google Cloud Storage) substitui o papel que o HDFS cumpria, sem exigir que a empresa monte e mantenha hardware próprio.
- O Apache Spark, ganhou popularidade como alternativa ou complemento ao MapReduce, sendo mais rápido e mais simples de usar.
- Importante: os princípios centrais do Hadoop (armazenamento distribuído, processamento paralelo, tolerância a falhas) continuam absolutamente válidos e presentes em toda arquitetura moderna de Big Data. O que mudou foram as ferramentas específicas que implementam esses princípios, não a lógica por trás delas.

2

BLOCO

Sistemas gerenciadores de bases não relacionais (NoSQL)

- Por que o modelo relacional não basta em escala
- ACID versus BASE e o Teorema CAP
- Os quatro modelos NoSQL
- Critério prático de escolha e polyglot persistence

Origem do NoSQL

1

Banco relacional (SQL)

Otimizado para dado estruturado, schema-on-write, forte consistência transacional (ACID), escala vertical.

2

Dois problemas quando volume, variedade e velocidade crescem

Dificuldade de escalar horizontalmente e rigidez de schema diante de dados semi e não estruturados.

3

Critério do NoSQL Archive (5 características)

Modelo não relacional; ao menos três dimensões do Big Data; sem schema fixo; escala horizontal massivamente distribuída; suporte a replicação.

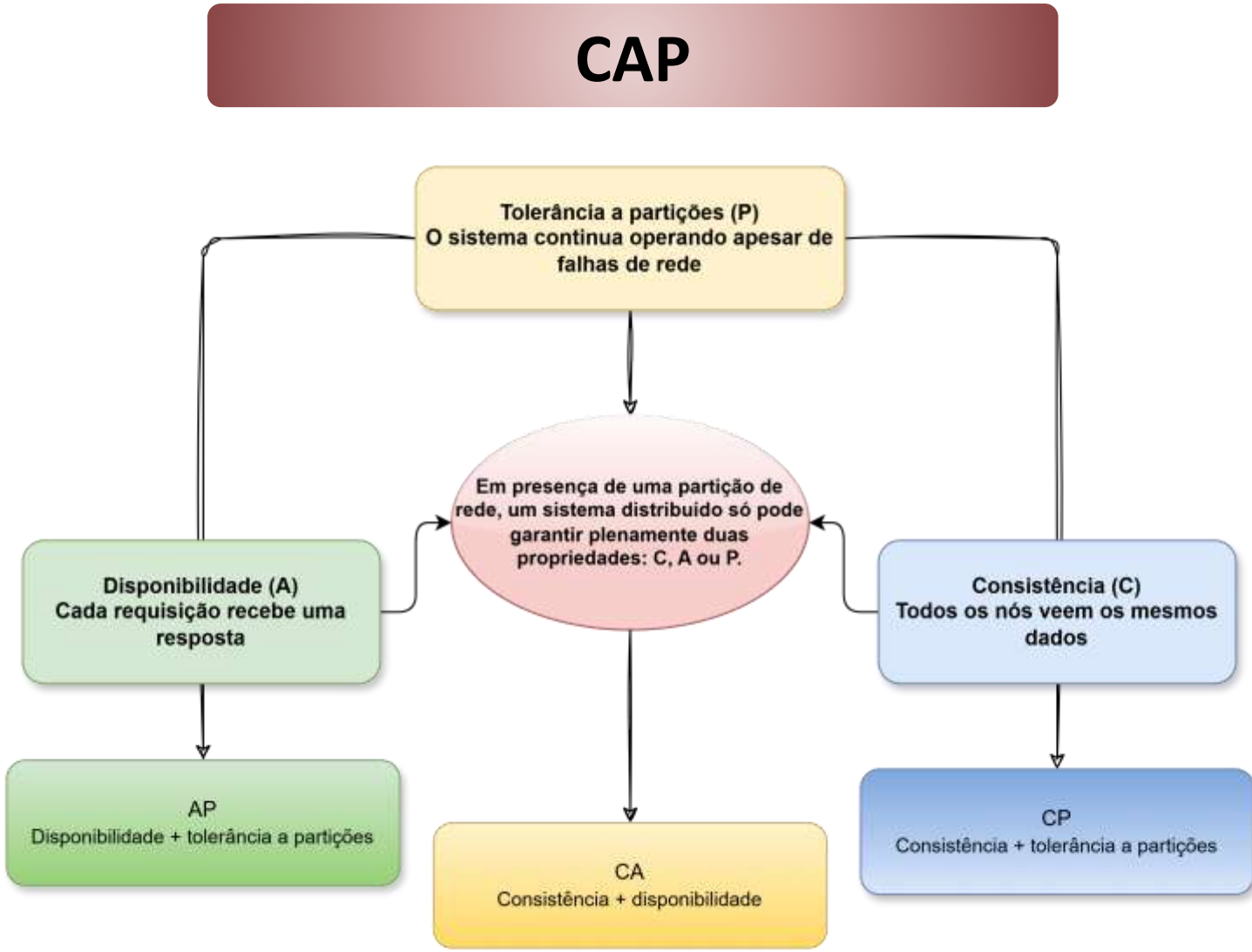
ACID , BASE e o Teorema CAP

ACID (padrão relacional)

- Atomicidade, Consistência, Isolamento, Durabilidade
- Garante estado consistente a cada transação

BASE (padrão NoSQL distribuído)

- Basically Available, Soft state, Eventual consistency
- Prioriza disponibilidade; consistência chega com atraso

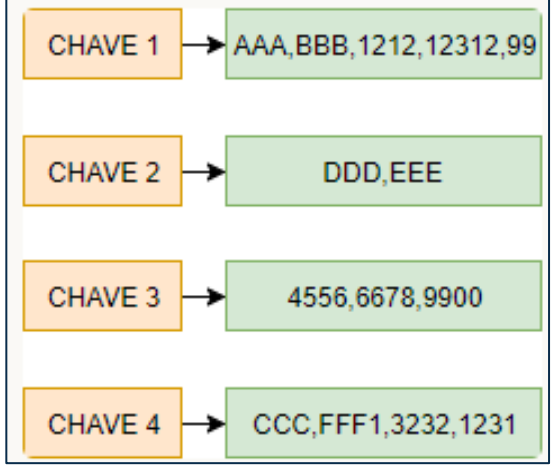


Modelos de bancos NoSQL

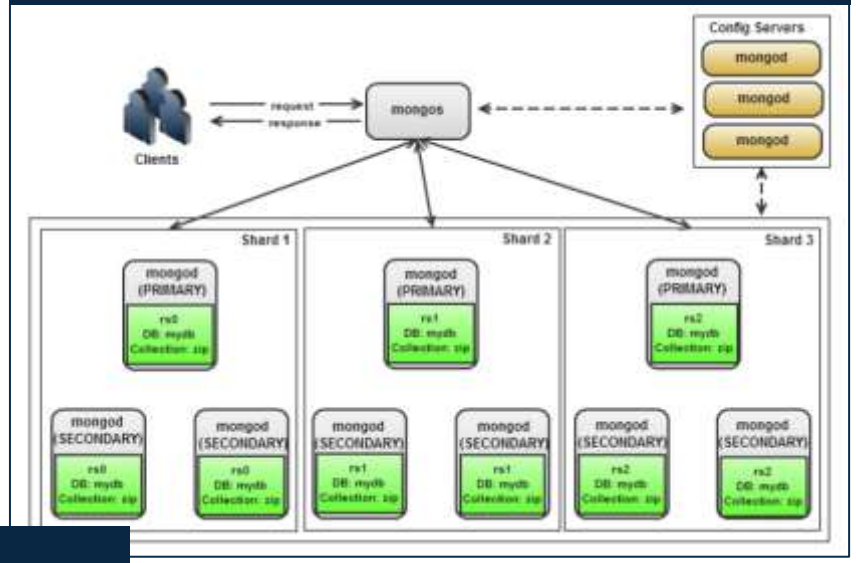
Modelo	Como organiza o dado	Exemplo de tecnologia	Cenário típico
Chave-valor	Par identificador + valor associado	Redis, DynamoDB	Sessão de usuário, carrinho de compras
Documento	Objetos JSON/XML, sem schema fixo	MongoDB, CouchDB	Catálogo de produtos, perfis variáveis
Colunar	Famílias de colunas, alto volume	Cassandra, HBase, Bigtable	Séries temporais, telemetria, logs
Grafo	Nós e arestas rotulados com propriedades	Neo4j	Rede de relacionamento, fraude, recomendação

Imagens dos modelos NoSQL

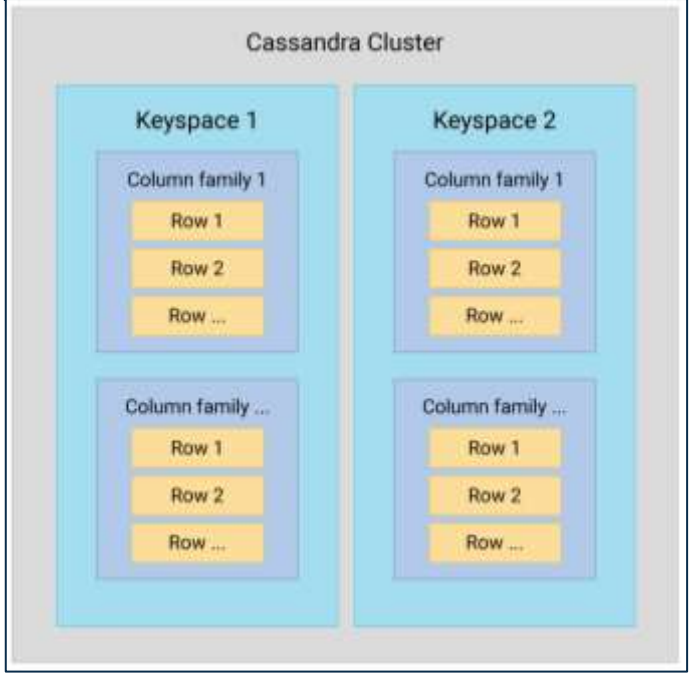
CHAVE-VALOR



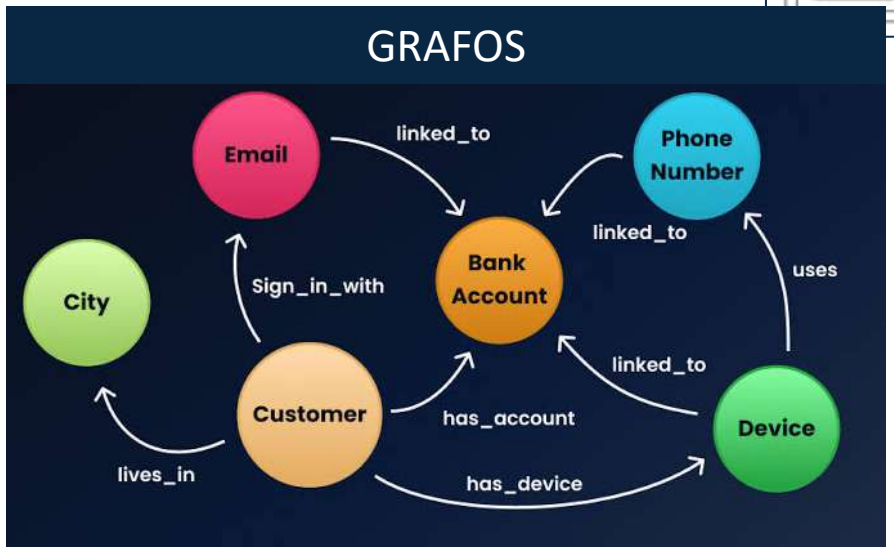
DOCUMENTO



COLUNAR



GRAFOS



EXEMPLO PRÁTICO DE DECISÃO

Telemonitoramento em escala: Relacional ou Colunar?

> *Para dezenas de milhares de eventos por segundo em toda a base de pacientes, um banco colunar como Cassandra é mais adequado que um banco relacional: a escrita é massiva e append-only, e a consulta típica é "todas as leituras de um paciente em um intervalo de tempo", exatamente o que esse modelo otimiza.*

Não existe o melhor banco de dados. Existe o banco adequado ao padrão de acesso e ao requisito de escala de cada caso de uso.

Use NoSQL quando houver:

- ✅ Grande volume de dados
- ✅ Crescimento rápido da aplicação
- ✅ Dados sem estrutura fixa
- ✅ Distribuição geográfica dos dados
- ✅ Necessidade de alta disponibilidade
- ✅ Aplicações web e mobile em larga escala

Prefira **bancos relacionais** quando houver:

- ✅ Muitas transações financeiras
- ✅ Forte necessidade de consistência
- ✅ Regras complexas de integridade
- ✅ Relatórios tradicionais altamente estruturados

Polyglot Persistence - Exemplo

1. **Relacional para dados financeiros e transacionais**

   Onde forte consistência e governança regulatória são essenciais.

2. **Documento para catálogos e conteúdo dinâmico**

   Onde a estrutura do registro varia de item para item.

3. **Colunar para telemetria em larga escala**

   Onde o padrão de escrita é massivo e a consulta é por intervalo de tempo.

Estudo de caso

Black Friday na VarejoMax

Contexto
A VarejoMax usa um banco SQL para catálogo, pedidos, sessões e recomendações. Na Black Friday, o volume e a variedade dos dados aumentam, causando dificuldades de escala horizontal e alterações frequentes no schema.

Desafio
A empresa precisa:

- manter pagamentos e pedidos com consistência transacional;
- sustentar milhares de carrinhos simultâneos;
- aceitar produtos com atributos variados;
- analisar grandes volumes de cliques e logs.

Pergunta:
Que combinação de bancos SQL e NoSQL você recomendaria? Justifique considerando **ACID, BASE e CAP**.

10

INTERVALO

De volta em 10 minutos

3

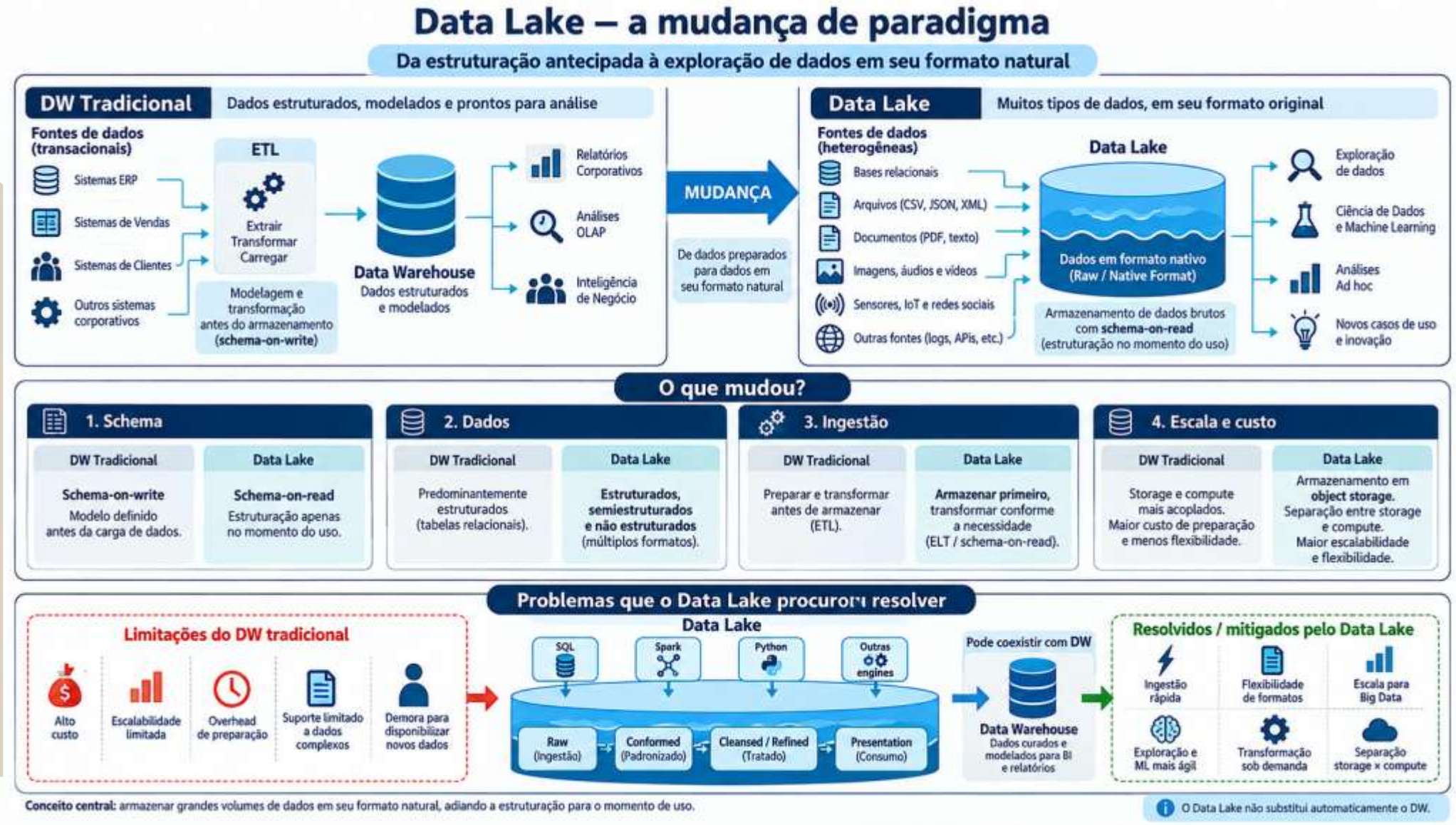
BLOCO

Arquiteturas modernas

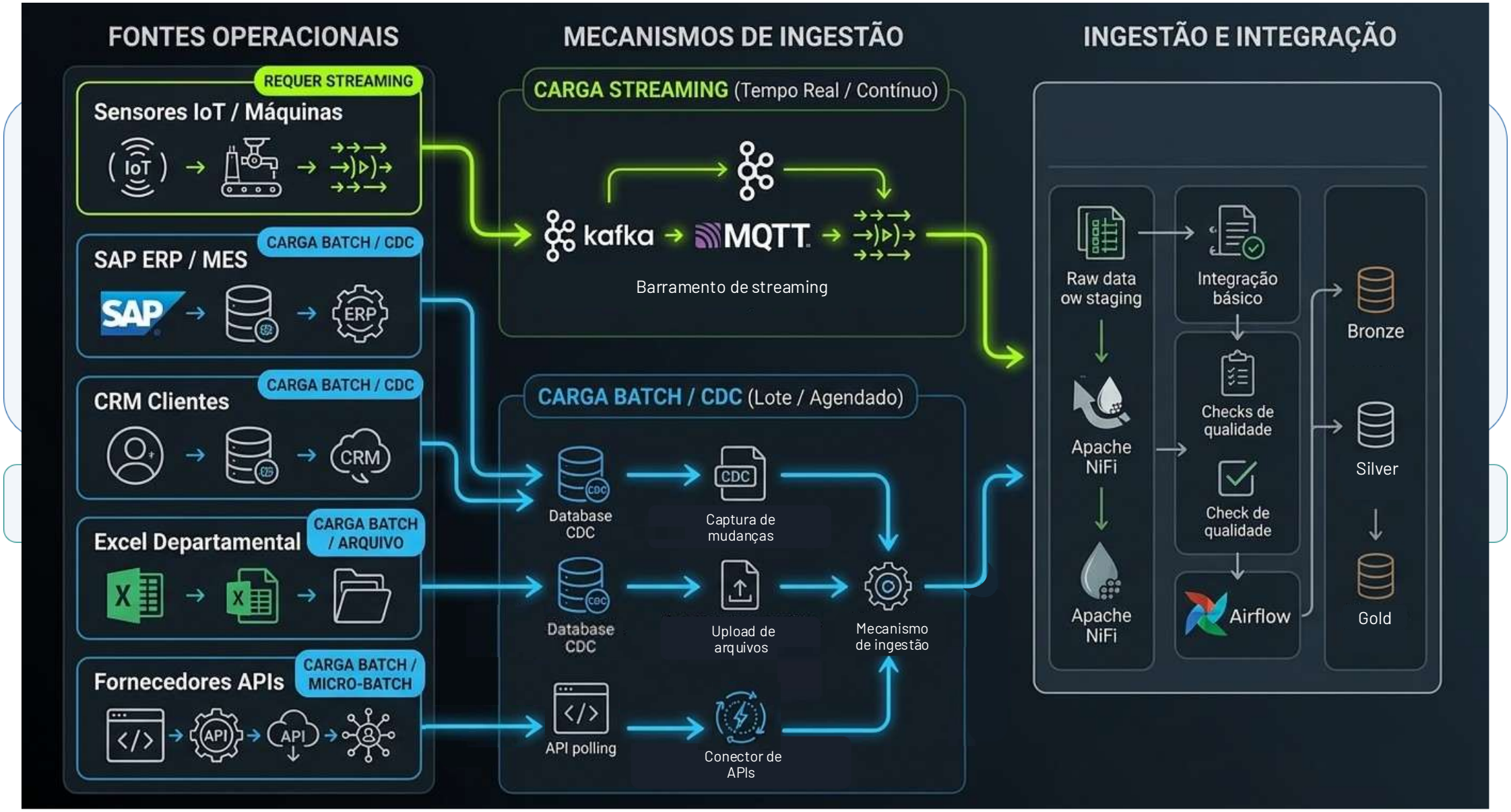
- Data Lake
- Arquiteturas de ingestão: Lambda & Kappa
- Lakehouse
- Quando considerar cada abordagem

James Dixon

- *Computação **distribuida***
  - ✓ *Armazenamento*
  - ✓ *Processamento*
- *Redes de alta velocidade*
- *Commodity Hardware*
- *Integração*



Fluxos Batch e Streaming



CENÁRIO

Atendimento Uber durante o Reveillon em Copacabana. A gestão da plataforma precisa:

- *Saber o que está acontecendo agora*

  | *“Há 2.350 motoristas disponíveis neste momento”*

- *Detectar carências*

  | *“Há bairros com demanda muito maior que a oferta”*

As 22h15, o sistema de processamento em tempo real falha durante 05 minutos.

O Sistema volta as 22h20 mostrando

| “Há 2.350 motoristas disponíveis”

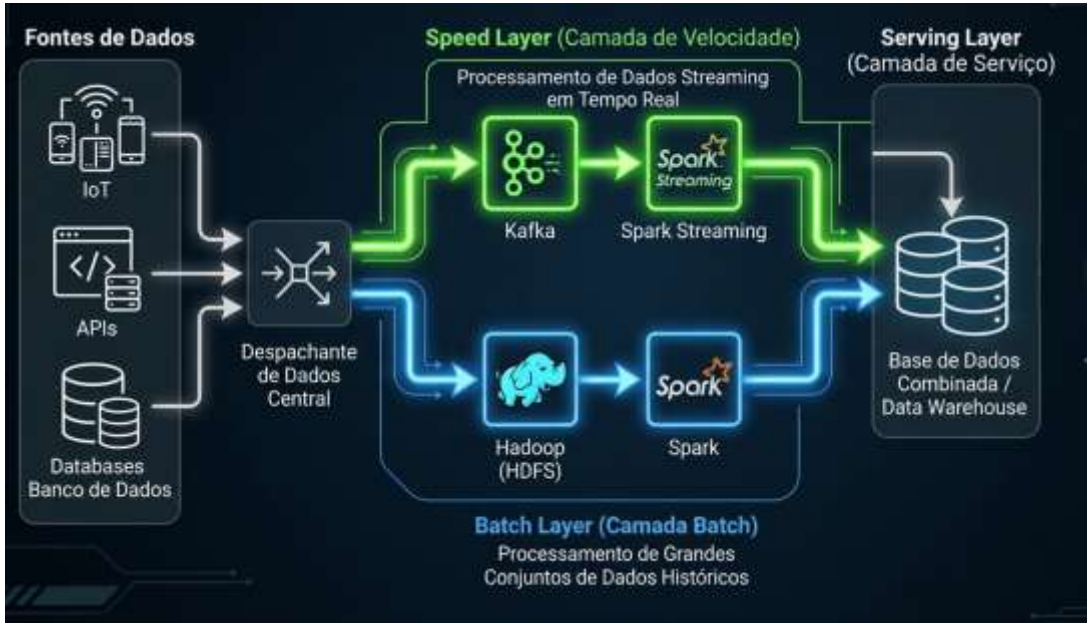
Mas o número correto é 2.000 motoristas disponíveis.

Processamento Batch e Streaming

ARQUITETURAS LAMBDA VS KAPPA

Arquitetura Lambda

Camada batch e camada de streaming rodam em paralelo; a camada de serviço une os resultados.

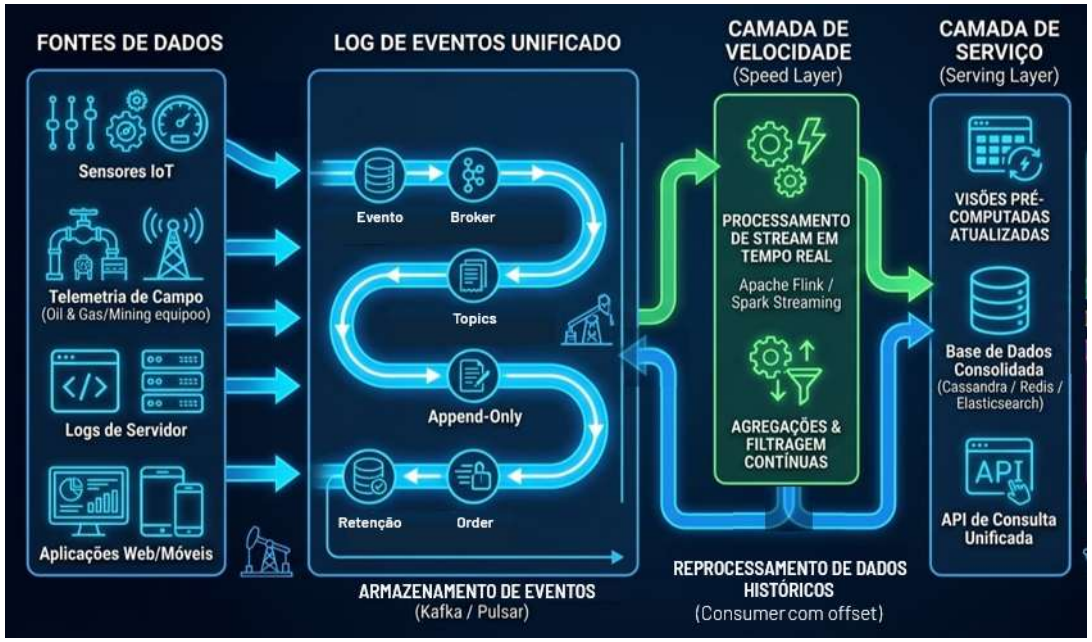


- **Batch Layer:** Processa dados históricos com alta precisão (ex: Hadoop/Spark).
- **Speed Layer:** Processa dados em tempo real com baixa latência (ex: Storm/Flink).

Desafio: Necessidade de manter duas bases de código duplicadas para os mesmos cálculos.

Arquitetura Kappa

Tudo é tratado como stream, inclusive o reprocessamento histórico.



- **Stream Unificado:** Tudo é tratado como um fluxo contínuo de eventos (Event Streaming via Apache Kafka/Pulsar).
- **Reprocessamento:** Feito simulando o replay do histórico através do log de eventos.

Vantagem: Simplificação arquitetural e código único para tempo real e histórico.

Estruturado e Não estruturado - Convivência

	Estruturado	Não estruturado
Raw	Guardado como chegou, sem alteração	Guardado como está, PDF ou imagem, intocado
Conformed	Convertido para Parquet	Só o metadado extraído vira Parquet
Cleansed	Schema validado, duplicidade tratada	Metadado ligado ao ativo correspondente
Presentation	Tabela pronta para BI	Tabela de metadados, com link para o arquivo

O arquivo original nunca vira tabela.
Quem vira estruturado é o metadado extraído sobre ele.

CAMINHO 1

O CSV atravessa as 4 camadas

Raw

O CSV chega e é guardado como veio, sem alteração.

Conformed

Convertido para Parquet, mesmo formato dos demais dados estruturados.

Cleansed

Schema validado, tipos corrigidos, duplicidade tratada.

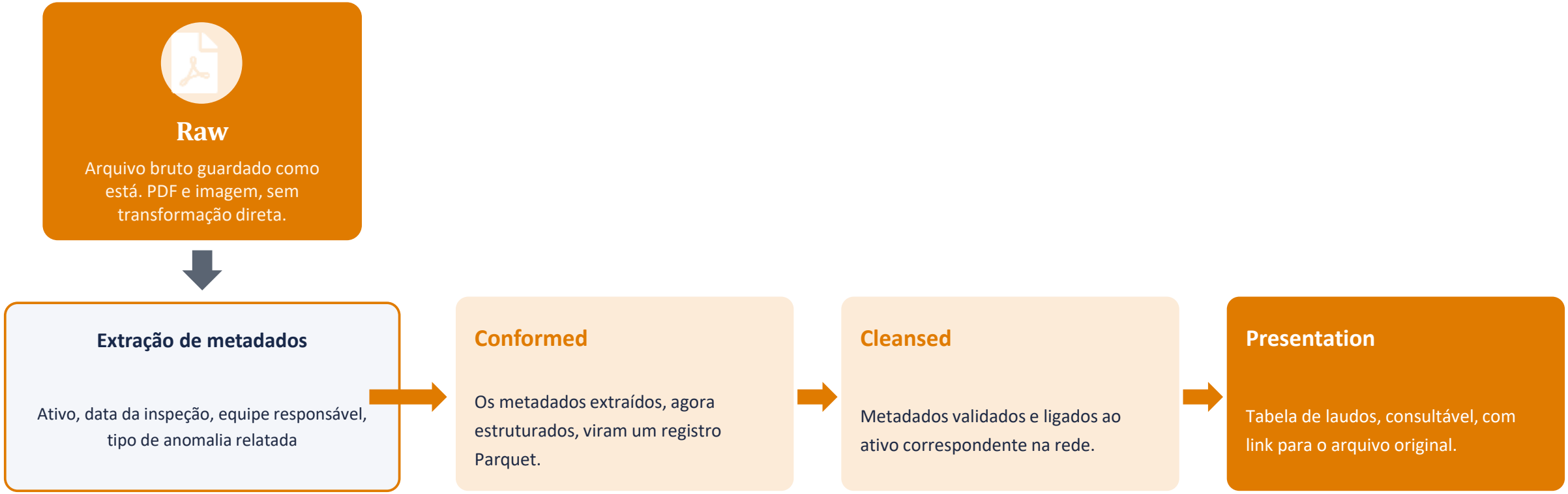
Presentation

Tabela pronta para BI.

Um caminho linear. Estrutura conhecida desde o início, cada camada só refina o que já existe.

CAMINHO 2

O JSON segue outro rumo



O arquivo original nunca vira tabela. Quem vira estruturado é o metadado extraído sobre ele.

2020

Lakehouse — a convergência entre Data Lake e Data Warehouse

Da arquitetura fragmentada ao Lakehouse

Arquitetura tradicional: Lake + Data Warehouse

Duas plataformas, mais complexidade

Fontes de dados (heterogêneas)

- Sistemas ERP / CRM
- APIs e aplicações
- Sensores / IoT
- Logs e arquivos
- Documentos, imagens, áudios, vídeos
- Outras fontes (dados externos)

Data Lake
Dados brutos (em seu formato original)

Armazenamento de baixo custo (schema-on-read)

ETL / ELT
Transformações e cópias de dados

Data Warehouse
Dados estruturados e modelados

- **BI / SQL** — Relatórios corporativos
- **Data Science / ML** — Exploração e modelos avançados

DUAS PLATAFORMAS = MAIS DESAFIOS

- Dados duplicados
- Pipelines adicionais
- Maior latência
- Inconsistência de dados

Lakehouse

Uma única plataforma para todos os workloads

Fontes de dados (heterogêneas)

- Sistemas ERP / CRM
- APIs e aplicações
- Sensores / IoT
- Logs e arquivos
- Documentos, imagens, áudios, vídeos
- Outras fontes (dados externos)

Lakehouse

Camada de tabelas e metadados (ACID, catálogo, governança)

Armazenamento em object storage (dados em formato aberto)

Uma única cópia lógica dos dados para múltiplos usos

- **BI / SQL** — Relatórios e dashboards
- **Engenharia de Dados** — ETL/ELT, streaming
- **Data Science / ML** — Modelos e experimentação
- **Analytics Avançado** — IA e aplicações de dados

O que mudou?

1. Armazenamento		2. Confiabilidade		3. Schema		4. Workloads		5. Governança	
Antes (Lake + DW)	**Lakehouse**	**Antes (Lake + DW)**	**Lakehouse**	**Antes (Lake + DW)**	**Lakehouse**	**Antes (Lake + DW)**	**Lakehouse**	**Antes (Lake + DW)**	**Lakehouse**
DW proprietário, armazenamento estruturado e de alto custo.	Formatos abertos (ex.: Parquet), object storage escalável e econômico.	Pipelines e cópias entre Lake e DW. Sem transações no lake.	Transações ACID, consistência, recuperação e confiabilidade sobre os dados do lake.	Schema rígido e preparação antecipada no DW.	Flexibilidade do lake (schema-on-read) + imposição e evolução de schema nas tabelas quando necessário (schema-on-write).	BI e SQL no DW. Data Science e ML no Lake. Plataformas separadas.	BI/SQL, engenharia de dados, Data Science e ML sobre a mesma base de dados.	Governança fragmentada entre plataformas. Metadados em sistemas distintos.	Catálogo unificado, segurança, auditoria e linhagem de dados sobre todos os dados e workloads.

Como o Lakehouse resolve

Problemas da arquitetura Lake + DW

- Duplicação de dados
- ETLs entre plataformas
- Dados desatualizados
- Custo e complexidade
- Data lock-in
- BI e ML separados

Formatos abertos (Parquet, etc.) · Delta Lake (ACID) · Metadados (catálogo) · Compute desacoplado do storage

- **Bronze** (Raw / Ingestão)
- **Silver** (Refinado)
- **Gold** (Business-ready)

Padrão Medallion: um padrão de implementação no ecossistema moderno (ex.: Databricks). Não é a definição original do conceito Lakehouse.

Resultados com o Lakehouse

- Uma cópia lógica dos dados
- Dados mais frescos
- BI + ML no mesmo ambiente
- Menor complexidade
- Escalabilidade
- Interoperabilidade (menos lock-in)

Lakehouse = economia e flexibilidade do Data Lake + confiabilidade, desempenho e usabilidade do Data Warehouse.

Conceito: Armbrust et al. (CIDR 2021) |
Implementação de referência: Databricks / Delta Lake / Unity Catalog.

Da necessidade ao componente físico

A decisão arquitetural começa no requisito, não no nome da plataforma.

MÉTODO

“Preciso receber pedidos novos sem recarregar tudo”

CDC / carga incremental

“Preciso atualizar registros existentes”

MERGE / upsert transacional

“Quero preservar exatamente o que chegou”

Bronze + object storage

“Quero saber quem alterou o dado”

histórico + lineage + audit

“Quero corrigir duplicidades”

Silver + regra de qualidade

“Quero repetir um processamento sem duplicar”

idempotência + checkpoints

↳ A habilidade avaliada é justificar a escolha: requisito → comportamento do pipeline → componente.

Arquitetura de referência

Uma implementação típica separa funções: armazenamento, tabelas, computação, catálogo, orquestração e consumo.



O Lakehouse não é um único produto: é a composição de capacidades que operam sobre object storage.

Como isso fica no armazenamento

A arquitetura lógica se materializa em diretórios, tabelas e metadados — com regras diferentes por camada.

STORAGE

bronze/orders raw/orders/

1 evento por arquivo ou lote

silver/orders silver/orders/

tabela confiável + dedupe + MERGE

gold/fato_vendas gold/fato_vendas/

grão definido pelo negócio

Regra prática de desenho

1. Bronze preserva o evento e facilita replay.
2. Silver concentra regras de confiança e integração.
3. Gold representa perguntas e métricas de negócio.
4. Metadados não devem ficar “escondidos” no nome da pasta.

↳ O diretório físico é consequência das regras de acesso, retenção, particionamento e processamento.

Erros de implementação que parecem pequenos

São decisões locais que podem gerar custo, retrabalho ou perda de confiabilidade.

ANTI-PADRÕES

1. **Gold como cópia da Silver**
   Sem modelar o consumo
2. **Micro-lotes sem compactação**
   Milhares de arquivos pequenos
3. **MERGE sem chave confiável**
   Updates imprevisíveis
4. **Schema livre em todas as camadas**
   Mudanças silenciosas
5. **Sem checkpoint/idempotência**
   Reprocessamento duplica dados
6. **Sem monitoramento de qualidade**
   Pipeline “verde” com dado errado

↳ *Use esta página como checklist de revisão antes da entrega do miniprojeto.*

Lakehouse × Data Warehouse: o mesmo caso, duas implementações

O comparativo deve ser feito pelo padrão de dados e pelas cargas de trabalho, não pela ideia de substituição automática.

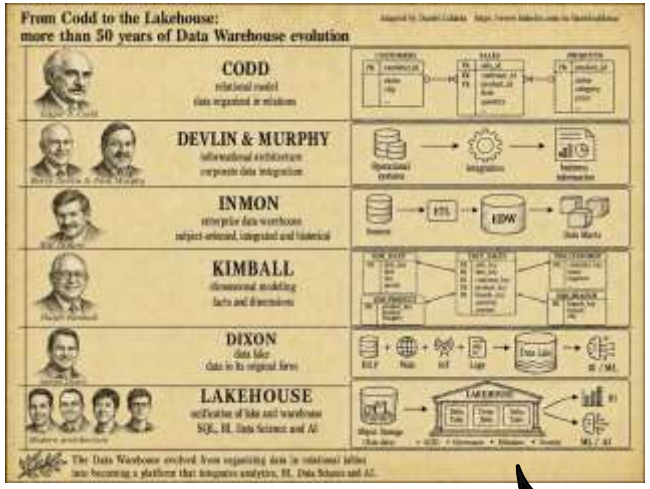
TRADE-OFF

Aspecto	Warehouse clássico	Lakehouse
Dados de entrada	tende a privilegiar dados modelados	aceita dados tabulares + eventos + arquivos
Atualização	ETL/ELT orientado a modelo	MERGE + tabelas transacionais no storage
Ciência de dados	possível, mas nem sempre é a camada primária	mesmo acervo pode atender analytics/ML
Governança	maturidade forte em ambientes tradicionais	precisa ser desenhada no ecossistema lake + catálogo
Caso forte	BI altamente estruturado e previsível	dados variados + analytics/ML + histórico no mesmo acervo

↳ A decisão real é “qual arquitetura reduz complexidade total para este caso?”.

Evolução por camada tecnológica

Resumo



Dimensão	Evolução
Armazenamento	Arquivos → tabelas relacionais → appliance/MPP → HDFS → object storage → tabelas transacionais abertas
Processamento	Programas sequenciais → SQL → ETL/OLAP → MapReduce → Spark → múltiplos engines
Modelagem	Registros físicos → modelo relacional → modelo corporativo → modelo dimensional → schema-on-read → camadas e produtos de dados
Integração	Arquivos batch → ETL → mensageria → streaming → CDC e pipelines incrementais
Consumo	Relatórios programados → OLAP → self-service BI → notebooks → BI, ML e IA integrados
Governança	Dicionário técnico → metadata repository → catálogo → linhagem e políticas centralizadas
Escalabilidade	Servidor central → SMP → MPP → clusters distribuídos → cloud elástica e separação entre compute e storage

Vamos pensar:

Retomando os conceitos vistos até aqui, justifique suas respostas.

1. **Data Lake e Lakehouse**
   Explique o armazenamento desacoplado e o papel dos formatos abertos. Como ACID e Time Travel ampliam as capacidades do Data Lake?

2. **Dimensões de Big Data**
   Escolha quatro dimensões apresentadas e exemplifique cada uma com autorizações, telemonitoramento ou laudos de uma operadora de saúde.

3. **Batch ou streaming?**
   Compare fechamento mensal de sinistros e bloqueio de transações suspeitas. Escolha o processamento adequado pelo custo da espera.

4. **Desafios de streaming**
   Explique como ordem, duplicidade e atraso dos eventos podem afetar um processamento contínuo. Relacione-os ao telemonitoramento.

5. **Lambda e Kappa**
   Compare as vias de processamento, o reprocessamento histórico e a manutenção de código. Em qual arquitetura o replay do log é central?

Estudo de caso

Estudos de Caso

Cenário adaptado Indústria de Petróleo e Gás

Disciplina 8 · Arquitetura de Dados e Big Data · Módulo 2

Os dois casos usam a mesma empresa fictícia, a PetroSul Exploração e Produção (PetroSul E&P), em dois momentos diferentes da sua evolução. O primeiro caso resolve um problema de armazenamento de dado em escala. O segundo caso parte do resultado do primeiro e resolve um problema de governança de dado entre áreas de negócio.

Estudo de Caso 1: Arquitetura NoSQL na PetroSul E&P

Contexto

A PetroSul E&P é uma empresa de exploração e produção de petróleo e gás de porte médio, com operações em plataformas offshore na costa brasileira. Hoje, cinco sistemas da empresa rodam sobre o mesmo banco de dados relacional único, e nos últimos 6 meses passaram a apresentar lentidão e picos de indisponibilidade. A diretoria pediu uma recomendação de arquitetura para cada sistema, com uma orientação clara: usar a ferramenta certa para cada problema, não trocar tudo por NoSQL só porque é tendência.

Sistema A. Monitoramento de Sensores IoT em Plataformas Offshore

Cerca de 500 poços produtores são instrumentados com sensores de pressão, temperatura, vibração e vazão em válvulas e bombas. Cada sensor envia uma leitura a cada 10 segundos, 24 horas por dia. A consulta mais comum é: "todas as leituras do poço X entre a data Y e a data Z", para gerar a curva de produção e detectar anomalias. Não há necessidade de alterar leituras antigas, só de adicionar novas.

Sistema B. Sessão e Cache do Aplicativo de Inspeção de Campo

O aplicativo é usado por técnicos de manutenção em plataformas, com conectividade instável via satélite. Guarda apenas um token de sessão, o checklist de inspeção em andamento e poucos atributos do técnico logado. Nos horários de troca de turno offshore, o sistema chega a 5 mil sessões simultâneas e precisa responder rápido, mesmo com rede instável.

Sistema C. Catálogo de Equipamentos e Peças de Manutenção

Os equipamentos variam muito entre si: válvulas, compressores, bombas e turbinas, cada um com atributos técnicos diferentes (torque para uma válvula, vazão nominal para uma bomba, por exemplo). Novos equipamentos e peças são cadastrados com frequência, exigindo mudanças rápidas na estrutura do dado, sem parar o sistema.

Sistema D. Investigação de Fraude e Conluio na Cadeia de Suprimentos

A auditoria interna quer identificar padrões suspeitos entre fornecedores, compradores internos e contratos: o mesmo fornecedor vencendo licitações de forma recorrente sempre com o mesmo comprador, ou fornecedores diferentes controlados pelo mesmo grupo econômico vencendo contratos de forma alternada. O que importa aqui não é o registro isolado, é a rede de relações entre fornecedores, compradores, contratos e CNPJs envolvidos.

Sistema E. Fechamento Financeiro Mensal de Produção e Royalties

Uma vez por mês, a empresa consolida o volume de produção e calcula os royalties devidos à ANP (Agência Nacional do Petróleo, Gás Natural e Biocombustíveis). Esse processo não admite valores parciais ou inconsistentes: o valor declarado precisa bater exatamente com o que foi auditado, sob risco de multa regulatória.

FGV

MÓDULO 2 CONCLUÍDO

Disciplina 8 · Arquitetura de Dados e Big Data — MBA em Inteligência Artificial e Analytics Aplicados a Negócios

Debora Corsino